

Structural Health Monitoring of Cantilever Beam, a Case Study—Using Bayesian Neural Network and Deep Learning



Rahul Vashisht, H. Viji, T. Sundararajan, D. Mohankumar
and S. Sumitra

Abstract The advancement of machine learning algorithms has opened a wide scope for vibration-based Structural Health Monitoring (SHM). Vibration-based SHM is based on the fact that damage will alter the dynamic properties, viz., structural response, frequencies, mode shapes, etc. of the structure. The responses measured using sensors, which are high dimensional in nature, can be intelligently analysed using machine learning techniques for damage assessment. Neural networks employing multilayer architectures are expressive models capable of capturing complex relationships between input–output pairs, but do not account for uncertainty in network outputs. A Bayesian Neural Network (BNN) refers to extending standard networks with posterior inference. It is a neural network with a prior distribution on its weights. Deep learning architectures like Convolutional Neural Network (CNN) and Long Short-Term Memory (LSTM) are good candidates for representation learning from high-dimensional data. The advantage of using CNN over multilayer neural networks is that they are good feature extractors as well as classifiers, which eliminates the need for generating hand-engineered features. LSTM networks are mainly used for sequence modelling. This paper presents both a Bayesian-multilayer-perceptron and deep-learning-based approach for damage detection and location identification in beam-like structures. Raw frequency response data simulated using finite element analysis is fed as input of the network. As part of this, frequency response was generated for a series of

R. Vashisht (✉) · H. Viji · S. Sumitra

Indian Institute of Space Science and Technology, Thiruvananthapuram 695547, India

e-mail: rahulvashisht290@gmail.com

H. Viji

e-mail: viji.narayan@gmail.com

S. Sumitra

e-mail: sumitra@iist.ac.in

H. Viji · T. Sundararajan · D. Mohankumar

Vikram Sarabhai Space Centre, ISRO, Thiruvananthapuram 695022, India

e-mail: t_sundararajan@vssc.gov.in

D. Mohankumar

e-mail: d_mohankumar@vssc.gov.in

© Springer Nature Singapore Pte Ltd. 2020

R. V. Prakash et al. (eds.), *Structural Integrity Assessment*, Lecture Notes in Mechanical Engineering, https://doi.org/10.1007/978-981-13-8767-8_64

simulations in the cantilever beam involving different damage scenarios (at different locations and different extents). These frequency responses can be studied without any loss of information, as no manual feature engineering is involved. The results obtained from the models are highly encouraging. This case study shows the effectiveness of the above approaches to predict bending rigidity with an acceptable error rate.

Keywords Bayesian neural network • Deep learning

1 Introduction

Deep learning models are being used on a daily basis to solve different tasks in vision, linguistics, and signal processing [1–5]. Understanding whether the model is under-confident or falsely overconfident can help get better performance out of the model. Recognising that test data is far from training data, one could easily augment the training data accordingly. In deep neural networks, usage of softmax to get probabilities is actually not enough to obtain model uncertainty. Standard neural network, with probability distribution over each of its weights is called BNN [6–11]. BNN gives the uncertainty estimates over the network outputs and can also help in model selection. Probabilistic backpropagation (PBP) is the learning technique used to train BNN in place of standard backpropagation.

This paper presents both a Bayesian-multilayer-perceptron-based approach and deep-learning-based approach (using architectures, viz., CNN and LSTM) for damage detection and location identification in cantilever beam. Raw frequency response data simulated using finite element analysis is fed as input of the network. Conventional data-driven approaches make use of statistical techniques for feature extraction from raw signals, in which case, data transformation has to be done accordingly for new data. The present approach models on raw data, which makes it ideal for real-time monitoring as it eliminates the need for data transformation. Details of each of the approaches are given in the following sections.

2 Experimental

2.1 *Bayesian Neural Networks*

Bayesian modelling is a powerful method to investigate the uncertainty in deep learning models [4]. Bayes theorem tells us about how to do inference from data. It follows the rule below to calculate conditional distribution of hypothesis given observed data.

$$P(\text{hypothesis}|\text{data}) = \frac{P(\text{data}|\text{hypothesis}) * P(\text{hypothesis})}{P(\text{data})}$$

Thus, learning and predictions can be seen as inference problems. Here, P (hypothesis) represents the prior beliefs over the hypothesis and $P(\text{data}|\text{hypothesis})$ represents likelihood of data given hypothesis which can be obtained from observed data. $P(\text{data})$ can be obtained by marginalising over the space of hypothesis and thus is considered as normalisation constant with respect to the hypothesis parameters.

Given training inputs $\mathbf{X} = \{x_1, x_2, \dots, x_N\}$ and their corresponding outputs $\mathbf{Y} = \{y_1, y_2, \dots, y_N\}$, in Bayesian (parametric) regression we would like to find the parameters ω of a function $y = f^\omega(x)$ that are likely to have generated the outputs. Following the Bayesian approach, some prior distribution is put over the space of parameters, $p(\omega)$. Prior distribution represents our belief as to which parameters are likely to have generated the data before we observe any data points. Once some data is observed the prior can be combined with the likelihood of the observed data to obtain the posterior which can be used for prediction of new data. The likelihood distribution $p(y|x, \omega)$ generates the output given the input and parameters ω . For regression, Gaussian likelihood is assumed which is as follows:

$$p(y|x, \omega) = N(y; f^\omega(x), \tau^{-1}I) \quad (1)$$

where τ is the model precision, which corresponds to adding observation noise to model output with variance τ^{-1} .

Using Bayes theorem, we can define the posterior as follows:

$$p(\omega|X, Y) = \frac{p(y|x, \omega)p(\omega)}{p(Y, X)} \quad (2)$$

This distribution captures the most probable function parameters given our observed data. For prediction of new input x^* , the posterior distribution and likelihood of new data x^* are combined and the parameters can be marginalised to obtain the predictive distribution for the new input, which is known as inference.

$$p(y^*|x^*, X, Y) = \int_{\omega} p(y^*|x^*, \omega)p(\omega|X, Y)d\omega \quad (3)$$

For most of deep learning models, the posterior distribution is intractable. Hence, posterior will be approximated by some approximating distribution $q(\omega|X, Y)$, which is obtained by minimising the KL-divergence between the posterior distribution and the approximating distribution [6].

Applying the Bayesian modelling gives the probabilistic interpretation of deep learning models by inferring distribution over model weights. Standard neural networks lack the capability of uncertainty measurement, as it only offers point

estimates. BNN offers robustness to overfitting, uncertainty estimates, and can easily learn from small datasets. We have used the probabilistic backpropagation to train the neural network which is discussed in the next section.

2.2 Probabilistic Backpropagation

Probabilistic backpropagation [8] (technique to train BNN) works similar to standard backpropagation (which is a technique to train a standard neural network). PBP uses a collection of one-dimensional Gaussians in place of point estimates for the weights, each one approximating the marginal posterior distribution of all the weights in the respective layer. In the forward pass, input data is propagated forward through the network. Since the weights are random, the activation produced in each layer is random, and results in intractable distributions. It sequentially approximates each of these distributions with the collection of one-dimensional Gaussians that match their marginal means and variances. At the end of this phase, PBP computes the logarithm of the marginal probability of the target variable instead of prediction error.

$$m^{a_l} = M_l m^{z_{l-1}} / \sqrt{n_{l-1} + 1}$$

$$v^{a_l} = [(M_l \circ M_l) v^{z_{l-1}} + V_l (m^{z_{l-1}} \circ m^{z_{l-1}}) + V_l v^{z_{l-1}}] / (n_{l-1} + 1)$$

where M_l and V_l are $n_l \times n_{l-1} + 1$ matrices whose entries are given by mean and variance of weight distribution. In the backward pass, the gradients of marginal distribution, Z is propagated backwards using reverse mode differentiation as in classical backpropagation. These derivatives are then used to update the means and variances of the posterior approximation.

The normalisation constant Z of posterior is obtained using approximation of integral as a normal distribution of the following form:

$$Z = N(y_n | m^{z_l}, v^{final})$$

where y_n is the actual output and m^{z_l} and v^{final} are mean and variance obtained after the forward pass, and v^{final} is variance obtained after forward pass and incorporating the prior information.

2.3 CNN

CNN [12] model can be described as the recursive application of convolution and pooling layer, followed by inner product layer, commonly called as dense layers, at the end of the network. Convolutional layer, which is the core layer of CNN is a

linear transformation that preserves the spatial information of the input space. Convolutional layer can be considered as a feature extraction layer. Pooling layers (optional) simply take the output of the convolutional layer and reduce its dimensionality. Intuitively, it is capable of bringing shift and distortion invariance to the input and to avoid overfitting to some extent. A convolutional layer's output shape depends on the shape of its input, shape of kernel, zero padding, and strides. The nonlinearity is incorporated in the network by applying activation functions such as relu, tanh, sigmoid, etc. after the convolution layer. A simple CNN, also known as ConvNet, is a sequence of layers. Every layer of a ConvNet transforms one volume of activations to another through a differentiable function. Convolutional layer does the convolution operation, which computes the output of neurons that are connected to local regions in the input. The pooling layer operates independently on every depth slice of the input and resizes it spatially. The layers used to build ConvNets are detailed below.

Convolutional Layer

This layer is the core building block of CNN. This layer does the convolution operation, which computes the output of neurons that are connected to local regions in the input. The CONV layer's parameters consist of a set of learnable filters. Every filter (also known as kernel) is small spatially (along width and height), but extends through the full depth of the input volume. During the forward pass, each of the filters is convolved with the input volume and dot products are computed between the entries of the filter and the input at any position. For the convolution layer, there are four hyperparameters, viz., number of filters K , the spatial extent of the filter F , the stride S , and the amount of zero padding P .

An input volume of size $W1 \times H1 \times D1$ produces a volume of size $W2 \times H2 \times D2$, where

$$W2 = (W1 - F + P)/S + 1$$

$$H2 = (H1 - F + P)/S + 1$$

$$D2 = K$$

The backward pass for a convolution operation is also a convolution (but with spatially flipped filters).

Pooling Layer

The pooling layer operates independently on every depth slice of the input and resizes it spatially. The depth dimension remains unchanged. The pooling options available are maximum pooling, average pooling, L2-norm pooling, etc. More generally, the pooling layer accepts a volume of size $W1 \times H1 \times D1$. The hyperparameters of this layer are their spatial extent F and the stride S . The resulting operation produces a volume of size $W2 \times H2 \times D2$, where

$$W2 = (W1 - F)/S + 1$$

$$H2 = (H1 - F)/S + 1$$

$$D2 = D1$$

The backward pass for a $\max(x, y)$ operation has a simple interpretation as only routing the gradient to the input that had the highest value in the forward pass. This layer is not mandatory and in some cases they are discarded.

Fully Connected Layer

Neurons in a fully connected layer have full connections to all activations in the previous layer, as seen in standard neural networks. The backward pass for FC layer can be done similar to CONV layer, where the spatial extent of the filter is the same as that of input volume.

Unlike traditional filters that have predefined parameters, the parameters of the 2D filter kernels in CNNs are automatically optimised by backpropagation.

2.4 LSTM

LSTM is a special kind of Recurrent Neural Network (RNN), capable of learning long-term dependencies [13]. They were introduced by Hochreiter and Schmidhuber. The core idea behind LSTMs lies that at each time step, a few gates are used to control the passing of information along the sequences that can capture long-range dependencies more accurately. LSTM has a chain of repeating modules of neural network, each having four neural network layers. The key to LSTMs is the cell state. The cell state runs straight down the entire chain, with only some minor linear interactions. LSTM consists of structures called gates, through which information is added or removed to the cell state. Gates are a way to optionally let the information through, composed out of a sigmoid neural network layer and a pointwise multiplication operation. At each time step t , hidden state h_t is updated by current data at the same time step, hidden state at previous time step, input gate, forget gate, output gate, and a memory cell. The decision regarding what information is to be through away from cell state is made by the forget gate layer. The forget gate accepts input at the current time step and hidden state at previous time step and outputs a number between 0 and 1 for each number in the cell state c_{t-1} . If the value is 0, it means to completely get rid of the information, while 1 means to completely retain (Fig. 1).

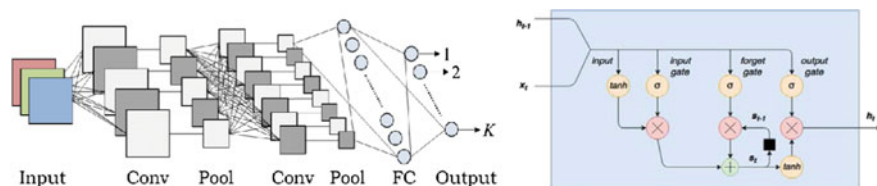


Fig. 1 Sample CNN [14] (left) and LSTM [15] architecture (right)

The basic LSTM equations are

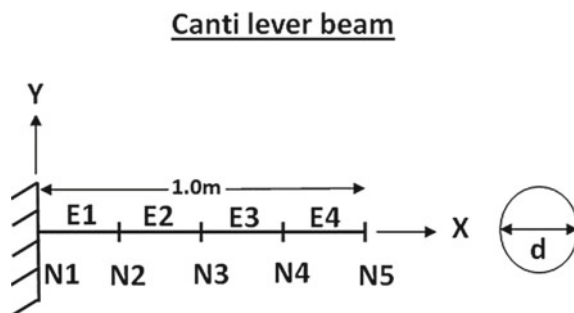
$$\begin{aligned}
 a_t &= \tanh(W_a x_t + U_a h_{t-1} + b_a) \\
 i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i) \\
 f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f) \\
 o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o) \\
 c_t &= f_t c_{t-1} + i_t \tanh(W_c x_t + U_c h_{t-1} + b_c) \\
 h_t &= o_t \tanh(c_t)
 \end{aligned}$$

where model parameters including all $W \in \mathbb{R}^{dxk}$, $U \in \mathbb{R}^{dxd}$ and $b \in \mathbb{R}^d$ are shared by all time steps and learned during model training, σ is the sigmoid activation function, denoting the element-wise product, k is a hyperparameter that represents the dimensionality of hidden vectors. The basic LSTM is constructed to process the sequential data in time order and the output at the terminal time step is used to predict the output.

2.5 Case Study

A cantilever beam structure of circular cross section having a length of 1 m is considered for the study. The structure is divided into four beam elements as shown in Fig. 2, where N stands for node and E stands for element [16–18]. Each element is having initial diameter of 0.01 m. Euler–Bernoulli beam element is used to idealise the structure. The nodal variables are transverse deflection and slope, where EI is the beam rigidity and l is the length of the element. The stiffness and mass matrix of the beam element are as follows:

Fig. 2 Cantilever beam



$$K = EI/l^3 \begin{bmatrix} 12 & 6l & -12 & 6l \\ 6l & 4l^2 & -6l & 2l^2 \\ -12 & -6l & 12 & -6l \\ 6l & 2l^2 & -6l & 4l^2 \end{bmatrix}$$

$$M = \rho A l / 420 \begin{bmatrix} 156 & 22l & 54 & -13l \\ 22l & 4l^2 & 13l & -3l^2 \\ 54 & 13l & 156 & -22l \\ -13l & -3l^2 & -22l & 4l^2 \end{bmatrix}$$

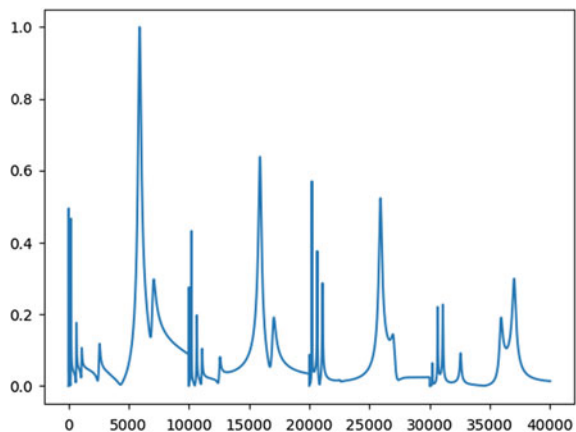
where ρ is the mass density, A is the area and l is the length of the element. Damage is simulated by changing the diameter of the individual elements. Frequency response analysis is carried out from 0.1 to 10,000 rad/s for 1N excitation by varying the diameter of the beam element from 0.005 to 0.015 m. The excitation force is applied at node number 5 in Y direction. The acceleration response is monitored at node numbers 2, 3, 4 and 5 in the Y direction.

3 Results and Discussion

Response from each node is of length 10,000. Here, a single data point is obtained by concatenating responses in the order 5, 4, 3, 2 forming a vector of length 40,000. Sample input is shown in Fig. 3, where x-axis represents four responses (where each ω is varying from 1 to 10000) and y-axis corresponds to normalised acceleration. The output for each data point is a vector of length 4 corresponding to four diameter of the elements E1, E2, E3 and E4. Total input data points are 14,641.

For training model using deep learning architectures, data was divided into training–testing with a split of 70:30. Training data was further split into training–

Fig. 3 Sample input of cantilever beam study, frequency versus acceleration



validation in the ratio 70:30. Adam was used as the optimiser. Mean squared error was used as the loss function.

3.1 CNN Model Architecture

To make the input data compatible for applying 2D CNN, each data point of length 40,000 is reshaped to 200×200 . To predict diameter for each element, four neural network models were developed. The number of convolution and pooling layers used in each network with the respective hyperparameters are given in Table 1. After each convolution layer, RELU [19] nonlinearity is applied. Only for the first convolution, zero padding is done for all models. The last two layers for all models are Global average pooling and Dense layer (with one neuron). The R-squared score for this model is 0.9862 (Fig. 4).

Table 1 Hyperparameters used in 2D CNN architecture for each beam element

	Conv1	MaxPool1	Conv2	MaxPool2	Conv3	MaxPool3	Conv4
Element E1	32 (3,3)	(4)	32 (3,3)	(2)	32 (3,3)	(2)	8 (3,3)
Element E2	64 (5,5)	(4)	32 (3,3)	(2)	32 (3,3)	–	–
Element E3	64 (5,5)	(4)	32 (3,3)	(2)	16 (3,3)	–	–
Element E4	16 (3,3)	(2)	32 (3,3)	(2)	–	–	–

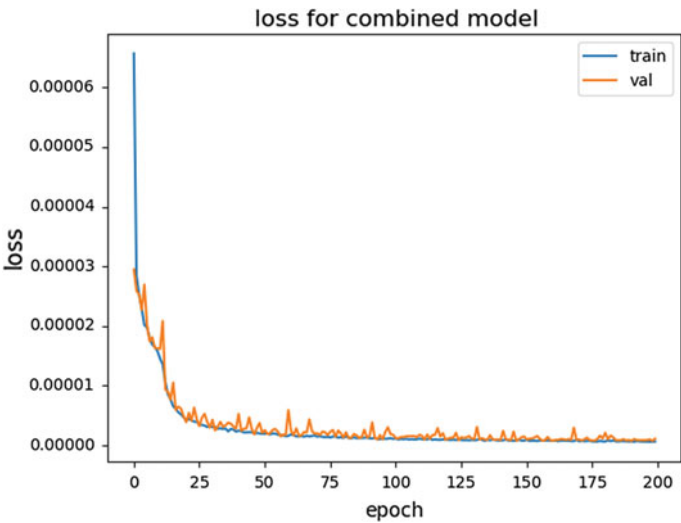


Fig. 4 2D CNN loss curve for cantilever beam

3.2 LSTM Model Architecture

To make the input data compatible for applying LSTM, each data point of length 40,000 is reshaped to (4, 10,000), where four represents the number of time steps. The data is modelled using a stack of three LSTM layers with 32, 16 and 4 hidden neurons, respectively. The loss curve is given in Fig. 5. The R-squared score for this model is 0.9910.

3.3 PBP Model Architecture

We have used the multilayer perceptron architecture with single hidden layer and 64 hidden units. It is trained using probabilistic backpropagation for 10 epoch with train test split of 50. The R-squared score for this model is 0.9967.

3.4 Model Evaluation

To evaluate the efficiency of the model, R-squared score is used as the metric. The R-squared metric provides an indication of the goodness of fit of a set of predictions to the actual values. It is a statistical measure that measures how close the data are to predictions. $R^2 = 1 - \text{SSE}/\text{SST}$, where SSE is the sum of squares of difference

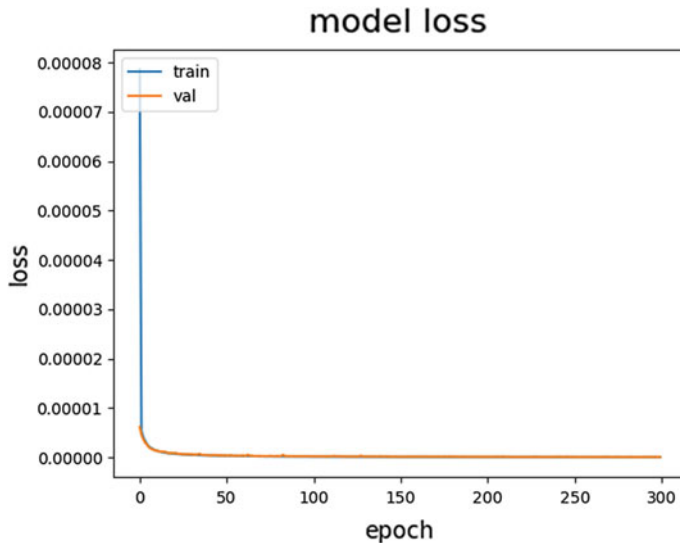


Fig. 5 LSTM validation loss for cantilever beam

Table 2 R-squared score of all architectures for cantilever beam

Algorithm	Test score
PBP	0.9967
LSTM	0.9910
2D-CNN	0.9862

between actual values and predicted values and SST is the sum of squares of difference between actual values and mean value.

R-squared statistic is defined as the percentage of the response variable variation that is explained by a linear model. It is always between 0 and 1. Here, we want our predicted values to be as close as possible to the actual values, and hence R-squared statistic should be close to 1. All the three models have given R-squared score, close to 1, which indicates the effectiveness of these algorithms to be applied on vibration-based SHM (Table 2).

3.5 Visualisation Using CNN Model

In regression outputs, one could visualise attention over input that increases, decreases or maintains the regressed filter index output. In this case study, this only tells us parts of the input that contribute towards to increase, decrease or maintaining the output value [20]. By default, Activation Maximisation loss yields positive gradients for inputs regions that increase the output and negative gradients that decrease the output. To visualise what contributed to the predicted output, gradients that have very low positive or negative values have to be considered. This can be achieved by performing $\text{grads} = \text{abs}(1/\text{grads})$ to magnifies small gradients.

A random test case taken for the study is shown in Fig. 5. The actual versus predicted output for the same is given in Table 3.

For our case study, we have to take top 10 gradients, which are shown in Figs. 6 and 7. The green lines show the top 10 features that maintain/contribute to the prediction. The blue lines show those features that can contribute to increase in the predicted value and orange lines are the ones that can contribute to decrease in the predicted value.

By visualising the gradients, it is evident that the first few modes are only required for identifying the damage near the fixed end whereas higher modes are required for identifying the damage near the free end, which is clearly reflected in gradient visualisation.

Table 3 Predicted and actual values for a test case of cantilever beam

	E1	E2	E3	E4
Actual diameter	0.011	0.008	0.015	0.015
Predicted diameter	0.01130249	0.00846803	0.01471727	0.01560052

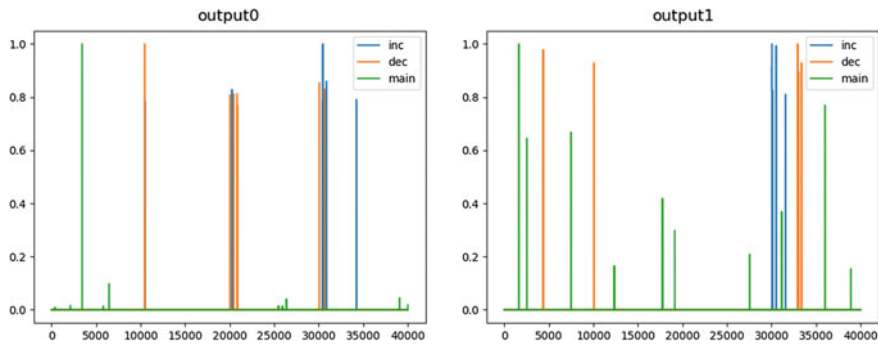


Fig. 6 Visualisation for outputs E1 and E2

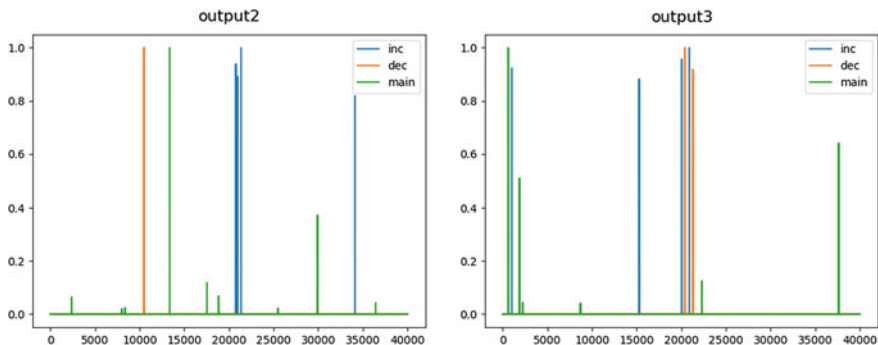


Fig. 7 Visualisation for outputs E3 and E4

4 Conclusions

The R-squared score obtained from all the three models are close to 1 for the unseen data, which shows that predictions are close to actuals and hence their generalisation capability. The R-squared score reflected by Bayesian model appears to outperform all the models. Moreover, gradient visualisation has demonstrated that neural networks are not just black boxes. It also gives us an intuition of the learning process, which closely associates with the underlying theory. The capability of deep learning architectures has shown their feature engineering capability, which could provide good predictions from raw vibration signals. This makes it ideal for real-time monitoring as it eliminates the need for data transformation.

Acknowledgements The authors would like to acknowledge Director, VSSC and Director, IIST for permitting us to carry out this study.

References

1. A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks. *Adv. Neural Inf. Process. Syst.* (2012)
2. J. Wu, *Introduction to Convolutional Neural Networks* (Nanjing University, China, 2017)
3. A. Karpathy, *Convolutional Neural Networks* (2016)
4. A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks. In: *Advances in Neural Information Processing Systems* (2012)
5. Y. LeCun, Y. Bengio, G. Hinton, Deep learning. *Nature* (2015)
6. J.M. Hernández-Lobato, R.P. Adams, Probabilistic backpropagation for scalable learning of Bayesian neural networks, in *Proceedings of the 32nd International Conference on Machine Learning*, vol. 37, ICML'15
7. S. Ghosh, F.M. Delle Fave, J. Yedidia, Assumed density filtering methods for learning bayesian neural networks, in *Disney Research, Association for the Advancement of Artificial Intelligence (AAAI)* (2016)
8. T.P. Minka, A family of algorithms for approximate Bayesian inference. Ph.D. thesis (2001)
9. R.M. Neal, Bayesian learning for neural networks. Ph.D. thesis (1995)
10. D.J.C. MacKay, Bayesian methods for adaptive models. Ph.D. thesis (1992)
11. A. Graves, Practical variational inference for neural networks, in *Advances in Neural Information Processing Systems* (2011)
12. K. Simonyan, A. Zisserman Very deep convolutional networks for large-scale image recognition. *CoRR* (2014)
13. Bearings fault diagnosis based on convolutional neural networks with 2D representation of vibration signals as input
14. https://www.researchgate.net/figure/An-example-of-CNN-architecture_fig1_320748406
15. <http://adventuresinmachinelearning.com/keras-lstm-tutorial/>
16. O. Abdeljaber, O. Avci, S. Kiranyaz, M. Gabbouj, D. Inman, Real-time vibration-based structural damage detection using one-dimensional convolutional neural networks. *J. Sound and Vib.* **388**, 154 (2017)
17. R. Zhao, R. Yan, Z. Chen, K. Mao, P. Wang, R.X. Gao, Deep learning and its applications to machine health monitoring: a survey. *arXiv preprint arXiv:1612.07640* (2016)
18. K. Simonyan, A. Vedaldi, A. Zisserman, Deep inside convolutional networks: visualising image classification models and saliency maps (2013)
19. V. Nair, G.E. Hinton, Rectified linear units improve restricted Boltzmann machines, in *Proceedings of the 27th International Conference on Machine Learning* (2010)
20. Y. Gal, Uncertainty in deep learning. Ph.D. thesis (2015)